

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Архитектура программных систем

Лабораторная работа №1

Выполнил:

Вильданов Ильнур Наилевич

Группа №Р3312

Преподаватель:

Перл Иван Андреевич

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

1. ЗАДАНИЕ	3
2. РАССМАТРИВАЕМАЯ СИСТЕМА.....	4
1. Система.....	4
2. Общее описание	4
3. Источники	6
3. ТРЕБОВАНИЯ К СИСТЕМЕ	7
3. System features	7
4. Nonfunctional requirements	11
5. External interface requirements	13
6. Requirements Attributes.....	15
7. Прецеденты	18
4. UML-ДИАГРАММЫ.....	28
5. ВЫВОД	32

1. ЗАДАНИЕ

Выбрать любую реально существующую систему и описать её в терминах UML. Желательно, чтобы система была не полностью информационной, но опиралась на информационную систему как показано в примере на лекции (Point of sale). Необходимо описать границы системы на разных уровнях, а также описать сценарии использования для нескольких Акторов.

Отчёт по работе должен содержать:

1. Титульный лист с указанием автора и номера группы
2. Само задание
3. Описание рассматриваемой системы с требованиями к ней
4. Формальное описание системы с необходимым количеством UML диаграмм
5. Словесное описание сценариев использования для рассматриваемых акторов

2. РАССМАТРИВАЕМАЯ СИСТЕМА

1. Система

Сервис аренды электросамокатов Whoosh - реальная городская система микромобильности, которая позволяет пользователю через мобильное приложение находить ближайшие самокаты, бронировать их, начинать и завершать поездки, а также оплачивать поездки через интегрированные платежные сервисы. При этом система опирается на информационную систему (приложение + серверные сервисы), но включает физические устройства (самокаты с электроникой и замком), что делает ее примером не полностью информационной системы.

2. Общее описание

2.1 Основные функции

Система обеспечивает полный жизненный цикл поездки: поиск самоката на карте, бронирование, начало поездки (сканирование QR-кода), временную паузу (блокировка колеса через приложение), завершение в разрешенной точке парковки и оплату с выдачей электронного чека. Дополнительно поддерживаются функции «Веер-beer» для поиска самоката и аренда нескольких самокатов на один аккаунт (групповая поездка).

2.2 Основные акторы

Система поддерживает несколько акторов:

- Неавторизованный пользователь (гость). Имеет доступ к просмотру карты и общих условий, но не может начать поездку.
- Авторизованный пользователь (клиент). Может бронировать самокаты, начинать и завершать поездки, ставить поездку на паузу, оплачивать и обращаться в поддержку.

- Служба поддержки. Принимает обращения пользователей в чате приложения или по e-mail и помогает завершить поездку/разобраться с инцидентами в рамках правил сервиса.

2.3 Операционная среда

Клиентская часть функционирует на мобильных устройствах iOS/Android и использует геолокацию, камеру (для сканирования QR), сетевое подключение и push-уведомления. Серверная часть взаимодействует с внешним платежным провайдером для блокировки депозита, списания оплаты и выдачи чеков. Физические самокаты принимают команды от системы (блокировка/разблокировка) и предоставляют состояние, необходимое для аренды.

2.4 Границы системы на разных уровнях

Чтобы явно зафиксировать границы, выделим три уровня описания:

- Бизнес-уровень: сервис аренды включает процессы поездок и взаимодействие с пользователем; не включает городскую инфраструктуру и поведение пользователей вне приложения.
- Уровень программной системы: включает мобильное приложение и серверные сервисы аренды; внешние платежные сервисы и служба поддержки рассматриваются как внешние участники.
- Уровень подсистем/компонентов: клиентское приложение, серверное API/сервисы и физическое устройство (самокат) с абстрактными интерфейсами взаимодействия (без привязки к конкретным протоколам).

2.5 Допущения и зависимости

2.5.1 Для корректной работы система должна иметь доступ к сети Интернет на мобильном устройстве пользователя.

2.5.2 Для аренды пользователь должен предоставить доступ к геолокации и камере (для сканирования QR-кода).

2.5.3 Платежные операции выполняются через внешнего платежного провайдера (блокировка депозита, списание оплаты, выдача чека).

2.5.4 Требования к парковке зависят от города и операционной модели; в городах, где используется парковка «Р», завершение поездки возможно только в точках, отмеченных на карте символом «Р».

3. Источники

Официальный FAQ Whoosh (англ.): https://whoosh.bike/faq_en

Карточка приложения Whoosh в App Store и Google Play (описание функций и требований): поиск по названию «Whoosh».

3. ТРЕБОВАНИЯ К СИСТЕМЕ

3. System features

3.1 Functional requirements

Registration and onboarding:

3.1.1 Система должна обеспечивать регистрацию пользователя с вводом и подтверждением e-mail и номера телефона.

3.1.2 Система должна обеспечивать добавление платежного метода (привязка карты напрямую или через Apple Pay/Google Pay).

3.1.3 Система должна обеспечивать подтверждение возраста пользователя; пользоваться сервисом должны только совершеннолетние.

3.1.4 Система должна предоставлять пользователю доступ к тексту пользовательского соглашения (оферты) и требовать подтверждения согласия при регистрации.

3.1.5 Система должна отображать пользователю актуальные тарифные параметры: базовый тариф региона и тариф выбранного самоката (стоимость старта, стоимость минуты, признак страхования).

Map and visibility:

3.1.6 Система должна отображать карту с доступными самокатами и парковочными точками «Р» (до и во время поездки).

3.1.7 Система должна отображать на карте зону катания и зону парковки, определенные для текущего города/региона.

3.1.8 Система должна показывать в карточке выбранного самоката параметры тарифа перед началом поездки: стоимость старта, стоимость минуты и наличие страхования.

3.1.9 Система должна показывать в приложении оценку остатка заряда самоката (в минутах и километрах) как приблизительную величину.

3.1.10 Система должна отображать на карте зоны ограничения скорости (restricted areas) и выделять их визуально.

Booking and finding a scooter:

3.1.11 Система должна позволять пользователю забронировать самокат на 10 минут; бронирование должно быть бесплатным.

3.1.12 Система должна отображать таймер бронирования и автоматически завершать бронирование по истечении 10 минут, делая самокат доступным другим пользователям.

3.1.13 Система должна позволять пользователю отменить бронирование из карточки самоката, подтверждая отмену.

3.1.14 Система должна блокировать депозит при начале бронирования (или перед стартом поездки) и не допускать бронирование при недостатке средств.

3.1.15 Система должна предоставлять функцию «Веер-beer» для поиска самоката (мигание фары и звуковой сигнал) и информировать, что фактическое положение может немного отличаться от метки на карте.

Trip start:

3.1.16 Система должна позволять начать поездку сканированием QR-кода на руле самоката или вводом имени/кода самоката (4 символа).

3.1.17 Система должна требовать, чтобы пользователь находился рядом с самокатом: для старта после бронирования необходимо нажать «Start» и отсканировать QR-код.

3.1.18 Система должна выдавать пользователю инструкции для старта движения (оттолкнуться и нажать кнопку GO на руле).

3.1.19 Система должна фиксировать согласие пользователя с текущей ценой при нажатии «Start trip» перед началом поездки.

3.1.20 Система должна обеспечивать начало поездки через приложение, включая перевод самоката в состояние «в поездке» и доступность управления (пауза/завершение).

During the trip:

3.1.21 Система должна позволять пользователю временно приостановить поездку и заблокировать самокат через интерфейс активной поездки (Lock scooter).

3.1.22 Система должна позволять разблокировать самокат из приложения (Unlock scooter) и продолжить поездку.

3.1.23 Система должна учитывать, что во время блокировки время аренды продолжает идти, стоимость увеличивается, а тариф в режиме блокировки не меняется.

3.1.24 Система должна поддерживать управление режимами скорости самоката (Sport/Drive/ECO) и информировать пользователя о текущем режиме.

3.1.25 Система должна автоматически снижать скорость в зонах с уменьшенным лимитом и информировать пользователя о наличии таких зон через карту.

Battery, parking and trip finish:

3.1.26 Система должна показывать оценку остатка заряда и отправлять push-уведомление при низком заряде с рекомендацией направляться к парковке.

3.1.27 Система должна учитывать, что при низком заряде самокат автоматически переходит в ECO-режим и едет медленнее.

3.1.28 Система должна позволять завершать поездку только в парковочных точках «Р» (в городах, где применяется модель парковки «Р»), отображаемых на карте.

3.1.29 Система должна требовать фотографию припаркованного самоката при завершении поездки для подтверждения корректной парковки.

3.1.30 Система должна фиксировать нарушения правил завершения поездки (например, отсутствие фото или парковка вне «Р») как основание для санкций/штрафа согласно правилам сервиса.

Rates, payments and multi-scooter:

3.1.31 Система должна рассчитывать стоимость поездки как сумму стоимости старта и стоимости минут, умноженной на длительность, плюс стоимость страхования (если пользователь не отключил его).

3.1.32 Система должна предоставлять пользователю возможность отключить страхование перед началом аренды/бронирования (если страхование включено по умолчанию).

3.1.33 Система должна поддерживать динамическое ценообразование в некоторых городах (изменение стоимости минуты до старта); при этом во время активной поездки тариф не должен изменяться.

3.1.34 Система должна использовать депозит для оплаты поездки в исключительных случаях (если списание с карты не удалось и стоимость не превышает депозит), а также обеспечивать разблокировку депозита после оплаты и выдачу чека/квитанции; при возврате должен формироваться отдельный чек.

3.1.35 Система должна позволять арендовать до 3 самокатов на один аккаунт одновременно, требуя депозит для каждого, завершение поездок по каждому самокату отдельно (с фото) и фиксируя ответственность владельца аккаунта за все арендованные самокаты.

4. Nonfunctional requirements

4.1 Usability requirements

4.1.1. Система должна быть рассчитана на использование с мобильных устройств; ключевые действия (старт/финиш поездки) должны выполняться не более чем за 3–4 нажатия.

4.1.2. Система должна поддерживать русский и английский языки интерфейса, а также обеспечивать корректное отображение текста при переключении языка.

4.1.3. Система должна предоставлять понятные сообщения об ошибках (например, самокат офлайн, недостаточно средств) и инструкции по дальнейшим действиям.

4.1.4. Система должна соответствовать базовым требованиям доступности: достаточный контраст, масштабирование текста, поддержка экранных дикторов на iOS/Android.

4.2 Reliability requirements

4.2.1. Система должна обеспечивать доступность серверной части не ниже 99.5% в сезон эксплуатации.

4.2.2. Система должна устойчиво обрабатывать временную потерю сети у клиента: поездка не должна «теряться», а состояние должно синхронизироваться при восстановлении связи.

4.2.3. Система должна вести журнал событий (audit log) для критичных операций: старт/финиш поездки, списание, возврат, смена статусов самокатов.

4.2.4. Система должна обеспечивать идемпотентность ключевых запросов (start/end ride, payment capture) для предотвращения дублей при повторах.

4.3 Performance requirements

4.3.1. Система должна обеспечивать загрузку карты и списка ближайших самокатов при нормальной сети не более чем за 2 секунды для 95% запросов.

4.3.2. Система должна обеспечивать подтверждение критических операций (например, старт/блокировка/завершение поездки) в пределах 10 секунд при штатной связи.

4.3.3. Система должна поддерживать горизонтальное масштабирование для роста числа активных поездок и подключенных устройств.

4.4 Security requirements

4.4.1. Система должна обеспечивать шифрование сетевого взаимодействия мобильного клиента с сервером (например, HTTPS с актуальными версиями TLS).

4.4.2. Система должна использовать токены доступа (например, JWT/opaque token) с ограниченным временем жизни и механизмом обновления.

4.4.3. Система должна обрабатывать платежные данные через сертифицированного платежного провайдера и не должна хранить полные номера платежных карт (PAN).

4.4.4. Система должна обеспечивать контроль прав доступа для всех внутренних административных функций и журналирование действий, выполняемых от имени служебных учетных записей.

4.4.5. Система должна иметь защиту от типовых атак на API (rate limit, защита от brute-force OTP, базовая валидация входных данных).

4.5 Legal and compliance requirements

4.5.1. Система должна хранить согласия пользователя с офертой/правилами и фиксировать версию документа, принятую пользователем.

4.5.2. Система должна обеспечивать соблюдение требований по персональным данным: минимизация, ограничение доступа, возможность удаления/анонимизации по запросу.

4.5.3. Система должна формировать электронные чеки и хранить их в истории поездок в соответствии с интеграцией с платежным провайдером.

5. External interface requirements

5.1 User Interfaces

5.1.1. Система должна предоставлять в мобильном приложении экран карты с отображением самокатов, парковок «Р» и зон ограничений.

5.1.2. Система должна предоставлять в мобильном приложении экран активной поездки: таймер, текущий тариф/стоимость, кнопки блокировки/разблокировки и завершения, а также доступ к правилам.

5.1.3. Система должна предоставлять в мобильном приложении экран платежей/чеков (получение квитанции по операции, в том числе по депозиту/возврату).

5.2 Hardware Interfaces

5.2.1. Система должна обеспечивать передачу команд управления самокатом (например, блокировка/разблокировка) и получение подтверждений/состояния устройства через доступные каналы связи.

5.2.2. Система должна обеспечивать получение от самоката статуса, необходимого для аренды (например, доступность для старта и низкий заряд) и корректное отражение этого статуса в приложении.

5.3 Software Interfaces

5.3.1. Система должна интегрироваться с платежным провайдером по API для операций блокировки депозита, списания оплаты и возвратов.

5.3.2. Система должна интегрироваться с картографическим сервисом (map provider) для отображения карты и нанесения объектов (самокаты, зоны, парковки).

5.3.3. Система должна поддерживать отправку push-уведомлений пользователю (например, при низком заряде или изменении статуса операции).

5.4 Communication Interfaces

5.4.1. Система должна предоставлять клиентскому приложению сетевой API и использовать структурированный формат данных (например, JSON).

5.4.2. Система должна обеспечивать надежную доставку команд и событий между компонентами (клиент ↔ сервер ↔ устройство), включая повтор при временных сбоях связи.

5.4.3. Система должна обеспечивать защиту сетевого взаимодействия (шифрование на транспортном уровне и аутентификацию запросов).

6. Requirements Attributes

Функциональные требования

№ требования	Приоритетность	Трудоемкость человеко-час	Стабильность
3.1.1	10	56	Высокая
3.1.2	9	24	Высокая
3.1.3	9	16	Высокая
3.1.4	10	12	Высокая
3.1.5	9	16	Средняя
3.1.6	10	64	Высокая
3.1.7	8	32	Средняя
3.1.8	10	40	Высокая
3.1.9	7	8	Высокая
3.1.10	6	12	Высокая
3.1.11	8	12	Средняя
3.1.12	8	24	Высокая
3.1.13	7	16	Высокая
3.1.14	10	32	Средняя
3.1.15	7	16	Средняя
3.1.16	10	40	Высокая
3.1.17	9	12	Высокая
3.1.18	7	8	Высокая
3.1.19	7	12	Средняя
3.1.20	9	32	Средняя
3.1.21	8	16	Средняя
3.1.22	8	20	Средняя
3.1.23	8	12	Средняя

3.1.24	7	24	Средняя
3.1.25	6	16	Средняя
3.1.26	6	12	Средняя
3.1.27	5	8	Средняя
3.1.28	8	20	Высокая
3.1.29	10	32	Высокая
3.1.30	9	24	Высокая
3.1.31	7	16	Средняя
3.1.32	10	24	Средняя
3.1.33	6	12	Средняя
3.1.34	8	24	Средняя
3.1.35	9	40	Средняя

Нефункциональные требования

№ требования	Приоритетность	Трудоемкость человеко-час	Стабильность
4.1.1	9	24	Высокая
4.1.2	8	16	Средняя
4.1.3	8	20	Высокая
4.1.4	7	16	Средняя
4.2.1	9	24	Высокая
4.2.2	8	20	Средняя
4.2.3	8	20	Средняя
4.2.4	7	16	Средняя
4.3.1	8	24	Высокая
4.3.2	7	16	Средняя
4.3.3	7	24	Средняя
4.4.1	10	20	Высокая

4.4.2	9	16	Высокая
4.4.3	8	16	Высокая
4.4.4	7	16	Средняя
4.4.5	7	12	Средняя
4.5.1	8	16	Высокая
4.5.2	7	20	Средняя
4.5.3	8	16	Средняя

7. Прецеденты

Прецедент: Просмотр карты и выбор самоката
ID: 1
Краткое описание: Пользователь открывает карту, видит доступные самокаты и выбирает подходящий.
Главный актер: Гость / Клиент
Предусловия: Приложение установлено; доступна сеть. Геолокация включена или пользователь выбрал район вручную.
Основной поток: 1. Пользователь открывает экран карты. 2. Система получает текущее местоположение (или выбранный район) и загружает список самокатов поблизости. 3. Система отображает самокаты и парковочные точки «Р», доступные в текущем городе. 4. Пользователь выбирает самокат на карте и открывает его карточку.
Альтернативный поток: Если геолокация недоступна, система предлагает включить ее или выбрать район вручную. Если самокатов рядом нет - предложить изменить масштаб/перейти в другую зону. Постусловия: Пользователь выбрал самокат или получил информацию об отсутствии доступных самокатов.

Прецедент: Регистрация пользователя
ID: 2
Краткое описание: Пользователь создает аккаунт, подтверждает контактные данные, добавляет платежный метод и подтверждает возраст.

Главный актер: Гость
Предусловия: Пользователь установил приложение и имеет доступ к сети.
Основной поток: 1. Пользователь открывает экран регистрации. 2. Система запрашивает e-mail и номер телефона и выполняет подтверждение. 3. Система предлагает добавить платежный метод (карта / Apple Pay / Google Pay). 4. Система запрашивает подтверждение возраста и предлагает прочитать правила/оферту. 5. Пользователь завершает регистрацию и попадает на главный экран.
Альтернативный поток: Если подтверждение контактов не удалось или платежный метод не добавлен - система не завершает регистрацию и показывает причину.
Постусловия: Аккаунт создан и готов к бронированию/поездке.

Прецедент: Просмотр оферты и подтверждение согласия
ID: 3
Краткое описание: Пользователь открывает пользовательское соглашение (оферту) и подтверждает согласие с правилами сервиса.
Главный актер: Клиент
Предусловия: Пользователь зарегистрирован; доступна сеть.
Основной поток: 1. Пользователь открывает раздел «About us/Offer» (или аналогичный раздел с документами). 2. Система отображает текст оферты/правил. 3. Пользователь подтверждает согласие с правилами (например, при

регистрации/перед первой поездкой).

4. Система фиксирует факт согласия и версию документа.

Альтернативный поток:

Если пользователь не подтверждает согласие - система не позволяет перейти к бронированию/поездке.

Постусловия: Согласие зафиксировано, пользователь может использовать сервис.

Прецедент: Бронирование самоката

ID: 4

Краткое описание: Пользователь бронирует самокат на 10 минут и при необходимости отменяет бронирование.

Главный актер: Клиент

Предусловия: Пользователь авторизован; добавлен платежный метод; на карте выбран самокат.

Основной поток:

1. Пользователь открывает карточку самоката и нажимает «Book».
2. Система блокирует депозит на счете пользователя.
3. Система устанавливает таймер бронирования на 10 минут и показывает обратный отсчет.
4. Пользователь либо подходит к самокату для старта поездки, либо отменяет бронирование.
5. Если 10 минут истекли - система завершает бронирование автоматически.

Альтернативный поток:

Если на карте недостаточно средств для блокировки депозита - система не выполняет бронирование и показывает сообщение.

Постусловия: Самокат забронирован на 10 минут или бронирование отменено/истекло и депозит разблокирован.

Прецедент: Начало поездки (сканирование QR)

ID: 5

Краткое описание: Пользователь начинает поездку: сканирует QR-код (или вводит код самоката) и переводит самокат в состояние «в поездке».

Главный актер: Клиент

Предусловия: Пользователь авторизован; платежный метод добавлен; пользователь находится рядом с самокатом (после бронирования или без него).

Основной поток:

1. Пользователь нажимает «Start» и сканирует QR-код на руле самоката (или вводит имя/код).
2. Система проверяет возможность старта (депозит, доступность самоката).
3. Система инициирует старт поездки и отображает экран активной поездки.
4. Система показывает инструкции: оттолкнуться и нажать GO на руле.

Альтернативный поток:

Если QR не читается - пользователь вводит код самоката вручную. Если депозит не заблокирован - старт запрещается.

Постусловия: Поездка начата, доступно управление (пауза/завершение).

Прецедент: Пауза поездки (Lock scooter)

ID: 6

Краткое описание: Пользователь временно ставит поездку на паузу, блокируя колесо самоката, и затем продолжает поездку.

Главный актер: Клиент
Предусловия: Поездка активна.
Основной поток: 1. Пользователь в экране активной поездки нажимает кнопку (желтая круглая) и выбирает «Lock scooter». 2. Система переводит самокат в режим ожидания (колесо заблокировано). 3. Пользователь возвращается и нажимает «Unlock scooter». 4. Система разблокирует самокат и возвращает поездку в активный режим.
Альтернативный поток: Если не удастся выполнить блокировку/разблокировку - пользователь оставляет самокат в безопасном месте и обращается в поддержку. Постусловия: Поездка временно приостановлена или продолжена; время аренды продолжает идти.

Прецедент: Завершение поездки и фото
ID: 7
Краткое описание: Пользователь завершает поездку в точке парковки «Р» и делает фотографию припаркованного самоката.
Главный актер: Клиент
Предусловия: Поездка активна; пользователь находится в парковочной точке «Р» (если модель парковок «Р» действует в городе).
Основной поток: 1. Пользователь подъезжает к ближайшей точке «Р», отмеченной на карте. 2. Пользователь нажимает кнопку в экране поездки и выбирает «End trip». 3. Система запрашивает фотографию самоката. 4. Пользователь делает фото и отправляет его.

5. Система завершает поездку и переходит к оплате.

Альтернативный поток:

Если пользователь не в точке «Р» - система не позволяет завершить поездку.

Если фото не отправлено - система фиксирует нарушение правил.

Постусловия: Поездка завершена; данные для расчета стоимости готовы; фото сохранено как подтверждение парковки.

Прецедент: Оплата, депозит и чек

ID: 8

Краткое описание: Система списывает стоимость поездки, разблокирует депозит и предоставляет электронный чек по операции.

Главный актер: Клиент

Предусловия: Поездка завершена; есть привязанный платежный метод.

Основной поток:

1. Система рассчитывает стоимость поездки по тарифу (старт + минуты + страхование).
2. Система списывает оплату с платежного метода пользователя.
3. Система разблокирует депозит после оплаты.
4. Система предоставляет пользователю чек/квитанцию по операции (и при возврате отдельный чек).

Альтернативный поток:

Если списание не удалось - система может использовать депозит для оплаты поездки, если стоимость не превышает депозит.

Постусловия: Оплата выполнена; депозит разблокирован; чек доступен пользователю.

Прецедент: Групповая поездка (до 3 самокатов)
ID: 9
Краткое описание: Пользователь арендует несколько самокатов с одного аккаунта для групповой поездки.
Главный актер: Клиент
Предусловия: Пользователь авторизован; платежный метод добавлен; первый самокат уже в активной поездке.
Основной поток: 1. Пользователь начинает поездку на первом самокате. 2. Система отображает в приложении кнопку «+» для добавления второго/третьего самоката. 3. Пользователь добавляет второй и (при необходимости) третий самокат и начинает поездку на каждом. 4. Система блокирует депозит отдельно для каждого самоката. 5. Пользователь завершает поездки по каждому самокату отдельно, включая фотографии.
Альтернативный поток: Если для очередного самоката не удастся заблокировать депозит - добавление самоката невозможно. Постусловия: До трех поездок активны одновременно; ответственность за все самокаты закреплена за владельцем аккаунта.

Прецедент: Обращение в поддержку
ID: 10
Краткое описание: Пользователь обращается в поддержку через чат в приложении или по e-mail, чтобы решить проблему (например, самокат не едет

или не удастся завершить поездку).
Главный актер: Клиент
Предусловия: Пользователь имеет активную поездку или столкнулся с инцидентом; доступна сеть (или есть возможность написать позже).
Основной поток: 1. Пользователь открывает чат поддержки в приложении (или пишет на support@whoosh.bike). 2. Пользователь описывает проблему и при необходимости прикладывает фото самоката/места. 3. Служба поддержки отвечает и дает инструкции (например, оставить самокат в парковке «Р» и завершить поездку позже).
Альтернативный поток: Если телефон разрядился - пользователь оставляет самокат в парковке и после восстановления доступа завершает поездку и сообщает поддержку для перерасчета. Постусловия: Обращение принято, пользователь получил инструкции/решение.

Прецедент: Поиск самоката через «Веер-beer»
ID: 11
Краткое описание: Пользователь не видит самокат на месте и включает сигнал «Веер-beer», чтобы найти его рядом с парковкой «Р».
Главный актер: Клиент
Предусловия: На карте выбран самокат; пользователь находится рядом с предполагаемым местом.
Основной поток: 1. Пользователь открывает карточку самоката на карте.

2. Пользователь нажимает «Веер-beer». 3. Система отправляет команду самокату подать сигнал (мигание фары и писк). 4. Пользователь находит самокат визуально/по звуку.
Альтернативный поток: Если самокат сместился - система информирует, что он обычно находится около ближайшей точки «Р», и предлагает поискать там. Постусловия: Самокат найден или пользователь понимает, что самокат недоступен/отсутствует.

Прецедент: Управление режимами скорости и обход restricted areas
ID: 12
Краткое описание: Пользователь переключает режим скорости самоката и планирует маршрут с учетом зон ограничения скорости.
Главный актер: Клиент
Предусловия: Поездка активна; самокат поддерживает режимы.
Основной поток: 1. Пользователь видит на карте выделенные зоны ограничения скорости (restricted areas). 2. Пользователь при необходимости планирует маршрут в обход зон, если не хочет ехать медленно. 3. Пользователь переключает режим скорости (Sport/Drive/ECO) на самокате. 4. Система информирует о текущем режиме и о том, что в restricted areas скорость снизится автоматически.
Альтернативный поток: Если заряд низкий - самокат автоматически переключается в ECO, система информирует об этом.

Постусловия: Пользователь контролирует режим скорости и учитывает ограничения на маршруте.

4. UML-ДИАГРАММЫ

1. Use-Case-диаграммы прецедентов использования, реализующих функциональные требования

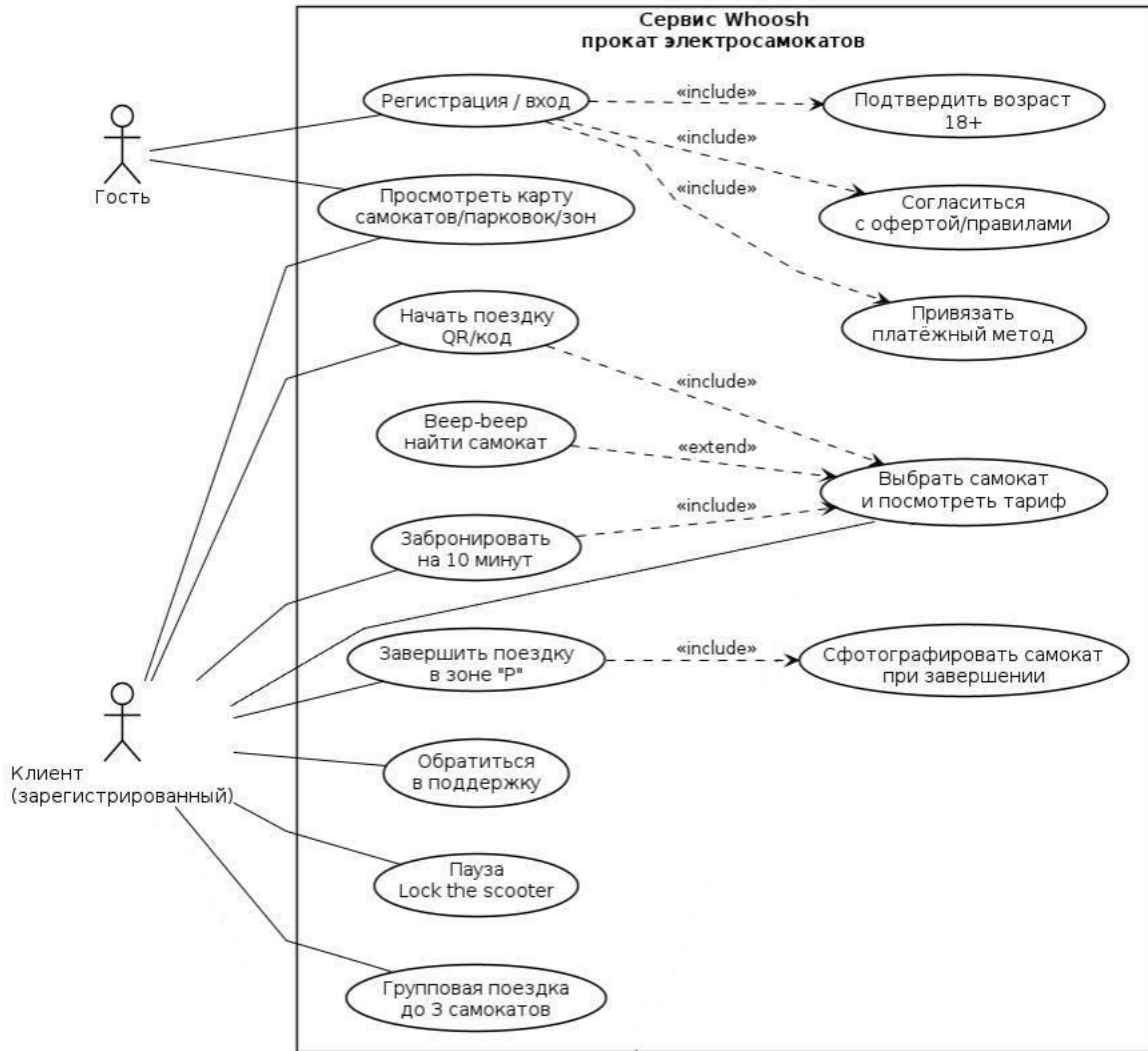


Рисунок 1. Use-Case пользователя (клиент)

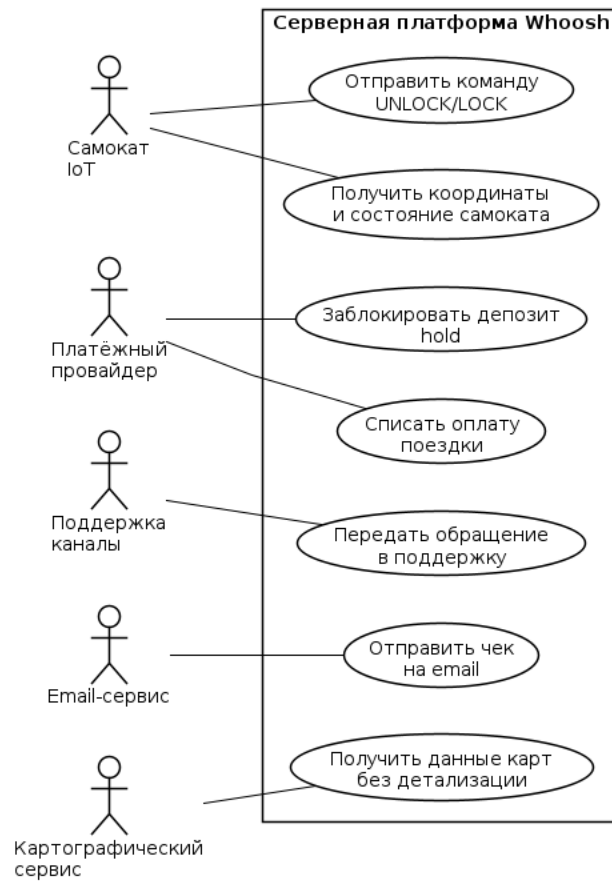


Рисунок 2. Use-Case внешних взаимодействий (самокат, платежи, поддержка)

2. Sequence-диаграммы

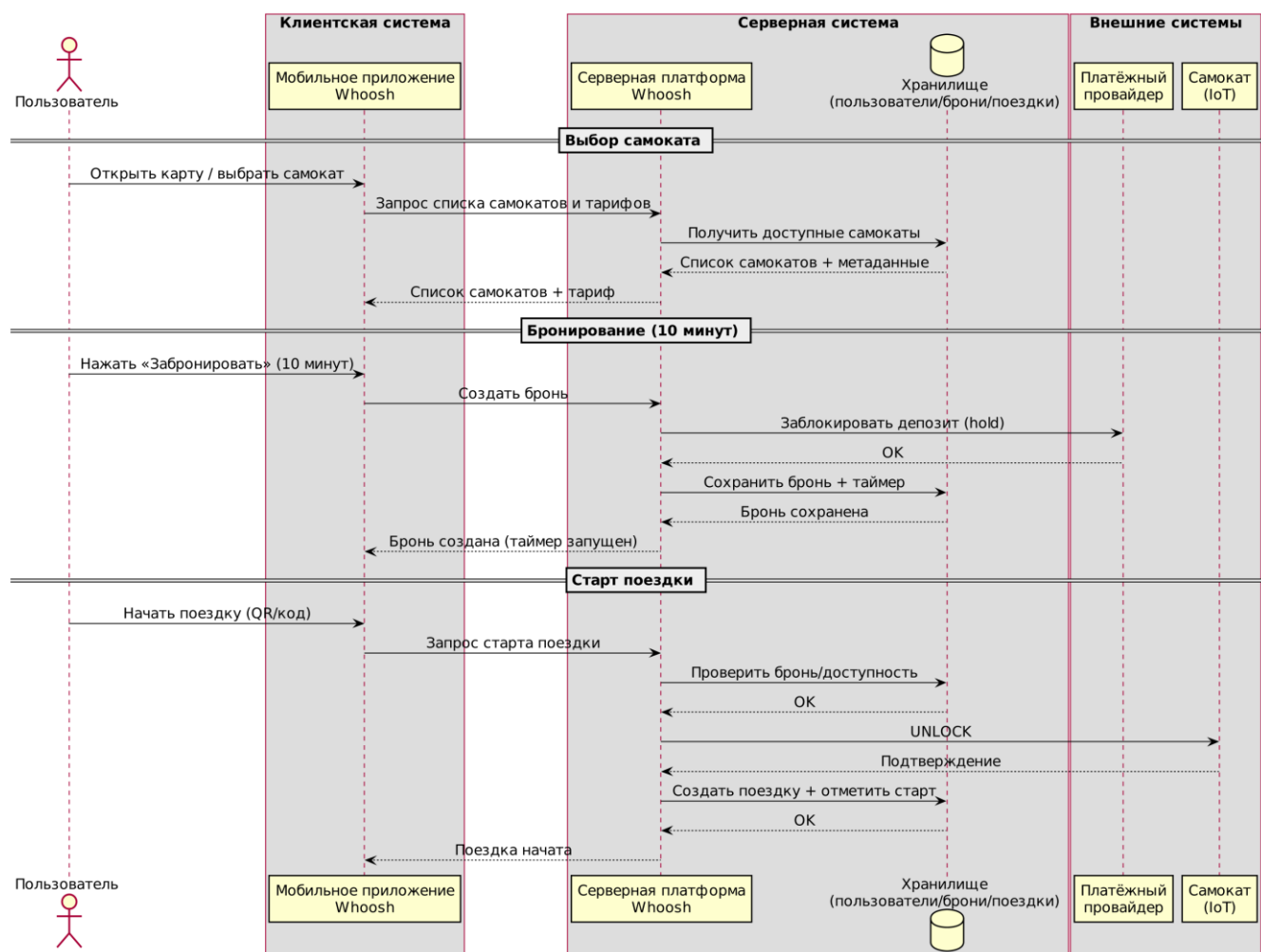


Рисунок 3. Основной поток: начало поездки

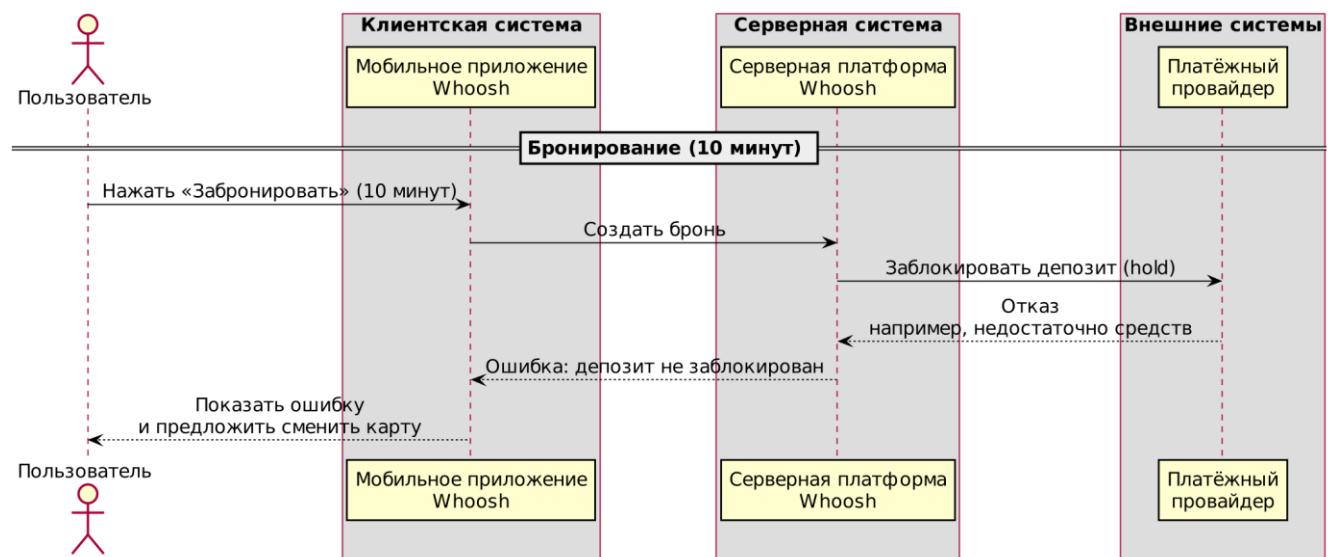


Рисунок 4. Альтернативный поток: депозит не заблокирован (недостаточно средств)

3. Компонентная диаграмма системы

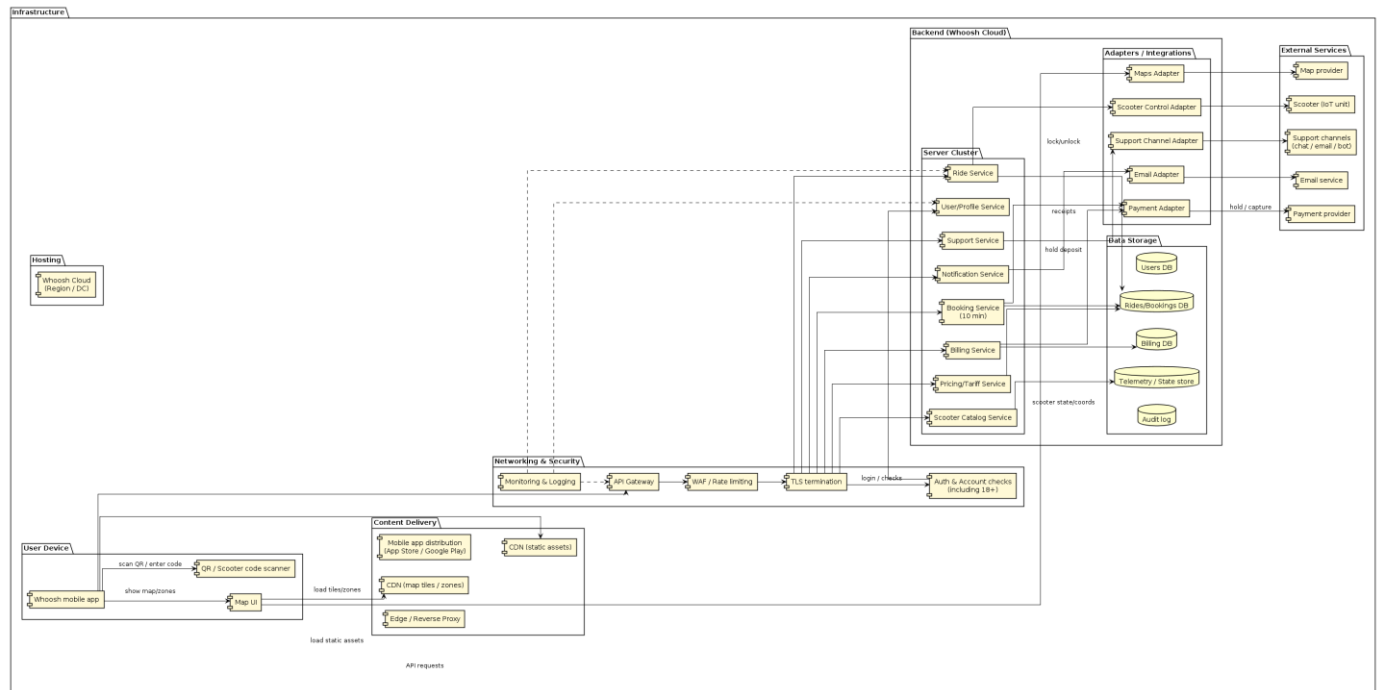


Рисунок 5. Компонентная диаграмма

5. ВЫВОД

В рамках лабораторной работы была выбрана реально существующая система аренды электросамокатов Whoosh и описана в терминах UML. Были определены границы системы на разных уровнях, ключевые акторы и сценарии использования, а также сформулированы функциональные, нефункциональные и интерфейсные требования. UML-диаграммы приведены как логическая модель системы, достаточная для анализа и проектирования.